Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

16 September 2026

gem5 TimingSimpleCPU · workload profile `test` · L1I/L1D 32 KiB, L2 256 KiB · gem5 `62d20b39c4+dirty`

Definitions. A `spill_store` is a stack store issued SP/FP-relative inside the region of interest; a `spill_load` is a load that reads a value back from the same address at the same access width, repeats counted. A `spill_event` is one (store, first read-back) pair, i.e. one *potential* spill. Hence `spill_event` ≤ `spill_store` (a value may be written and never read back again inside the region) and `spill_load` ≥ `spill_event` (a value may be read back many times), while `total_spill_insts` = `spill_store` + `spill_load`. The classification is microarchitectural and cannot separate compiler register spills from ABI-mandated callee-save traffic or ordinary scalar stack locals, so these are upper bounds on true register-allocator spilling; the store side is the looser of the two, because at store time a later read-back is unknowable. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span exactly the same window, and the ROI instruction, load and store columns were cross-checked against gem5's own `simOps` and `commitStats0` counters. *Spill %* is `total_spill_insts/ROI_total_insts`.

Benchmark	ROI_total_insts	ROI_load	ROI_store	spill_store	spill_load	total_spill	**spill_event**	Spill %	L1D hits	L1D miss	L1D m %	L2 hits	L2 miss	L2 m %	sp_load m %	sp_store m %
gcc	16 026 850	2 957 472	1 955 133	1 103 563	1 041 215	2 144 778	1 020 580	13.38	4 826 290	86 315	1.757	111 297	57 599	34.10	0.012	0.055
Total	16 026 850	2 957 472	1 955 133	1 103 563	1 041 215	2 144 778	**1 020 580**	13.38	4 826 290	86 315	1.757	111 297	57 599	—	—	—